DevSecOps Interview Preparation Guide

Azure and AKS | Kubernetes | Terraform | GitHub Actions | CI/CD | Container Security | Troubleshooting

This is a practical study reference covering fundamentals, common interview questions with short answers, commands used during deployment and incident response, and hands-on scenarios. Examples are educational; always use approved credentials, environments and change controls when practicing.

1. Understand the complete code-to-production flow

Stage	What happens
1. Plan and code	Create a ticket, branch from the agreed baseline, write code and tests, follow secure-coding conventions.
2. Commit and pull request	Push to Git, open a pull request, require review, enforce branch protection and check for exposed secrets.
3. Continuous integration	Trigger GitHub Actions/Jenkins/GitLab CI; compile or build, run unit tests, lint, SAST and dependency checks.
4. Build trusted artifact	Build an immutable container image, scan dependencies and image, create an SBOM if required, publish to ACR/ECR.
5. Infrastructure as code	Run Terraform formatting, validation, security checks and plan; review changes before applying to the target environment.
6. Release and deploy	Use immutable image digests and Helm/Kubernetes manifests; obtain Azure access through federated identity where possible.
7. Validate	Wait for rollout, check readiness, smoke-test APIs, observe logs/metrics and compare with service objectives.
8. Operate and improve	Monitor, alert, respond to incidents, roll back where safe, document root causes and improve security controls.

Explain this flow aloud: Source -> pull request -> automated tests and scans -> signed/versioned image -> registry -> deployment approval -> AKS/Kubernetes -> smoke tests -> monitoring and rollback. Distinguish CI (verify/build) from CD (release/deploy).

2. CI/CD, Git and release fundamentals - questions and answers

1. What is CI, and what is CD?

Answer: Continuous integration automatically builds and tests integrated code. Continuous delivery keeps a release ready for approved deployment; continuous deployment releases validated changes automatically.

2. What are typical CI pipeline stages?

Answer: Checkout, dependency installation, lint/static checks, unit tests, application build, security scans, artifact or image packaging, and publishing a versioned artifact.

3. What is the difference between a pipeline artifact and a container image?

Answer: An artifact may be any output, such as a ZIP or JAR; a container image packages application code and filesystem/runtime layers for container execution.

4. What is the purpose of a pull request and branch protection?

Answer: To enable review, automated checks and controlled merging. Protect important branches against unreviewed or failing changes.

5. What is the difference between Git merge and rebase?

Answer: Merge joins branch histories, usually with a merge commit; rebase replays commits onto a different base and rewrites local commit ancestry. Avoid rebasing shared history without coordination.

6. How do you version and promote the same release?

Answer: Build once; tag by commit SHA or immutable digest; promote the same artifact to test, staging and production without rebuilding from different source.

7. How do you roll back a bad production release?

Answer: Identify the last healthy release, verify database/schema compatibility, restore the previous image/chart or deployment revision, then confirm metrics, logs and smoke tests.

3. GitHub Actions - questions and answers

8. What is a GitHub Actions workflow?

Answer: A YAML automation definition in .github/workflows. Events such as push or pull_request start one or more jobs; each job contains steps running on a runner.

9. Explain workflow, job, step, action and runner.

Answer: A workflow is the automation; a job is a group of steps; a step executes an action or shell command; an action is a reusable task; a runner is the machine executing the job.

10. What is the difference between needs and if?

Answer: needs defines job dependencies and ordering; if conditions execution based on expressions or results.

11. How do you protect production deployments in GitHub Actions?

Answer: Use protected environments, required reviewers, restricted branches, concurrency controls, least-privilege permissions, and a deployment job that runs only after tests/scans and approval.

12. How should GitHub Actions authenticate to Azure?

Answer: Prefer GitHub OIDC federation with a Microsoft Entra application or managed identity and narrowly scoped Azure permissions; avoid long-lived client secrets.

13. How do you pass build outputs between jobs?

Answer: Use explicit job outputs for small values and upload/download artifacts for files; do not rely on one job retaining another job's local filesystem.

14. What is the difference between secrets and vars in GitHub Actions?

Answer: Secrets contain sensitive values and are masked in logs where possible; variables store non-sensitive configuration. Prefer short-lived cloud tokens to static secrets.

15. How do you avoid duplicated or overlapping deployments?

Answer: Use concurrency groups, environment protection and appropriate cancellation behavior so only one controlled deployment targets a given environment at a time.

Minimal GitHub Actions CI example (Node.js application)

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
permissions:
  contents: read
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: 'npm'
      - run: npm ci
      - run: npm test -- --ci
      - run: npm run build
```

Adapt commands to the application stack. Add SAST/SCA, image scanning, build provenance, artifact publishing and separate guarded deployment jobs for production. Pin third-party actions to reviewed commit SHAs for stronger supply-chain assurance.

4. Azure and AKS - questions and answers

16. What is Azure Kubernetes Service (AKS)?

Answer: A managed Kubernetes service in Azure. Azure manages the control plane; customers still configure node pools, workloads, networking, identities, policies, upgrades and observability.

17. How do Azure subscriptions, resource groups and VNets differ?

Answer: A subscription is a billing and governance boundary; a resource group logically groups Azure resources; a VNet provides private IP connectivity and subnets.

18. What is the difference between an AKS system node pool and user node pool?

Answer: System pools host critical system components; user pools host application workloads and can be sized, tainted and scaled for application requirements.

19. What is a private AKS cluster?

Answer: Its Kubernetes API server uses private connectivity instead of an internet-exposed endpoint; administrators and automation require appropriate network and DNS access.

20. What is Azure Container Registry (ACR) used for?

Answer: Storing and distributing OCI container images/artifacts, with controls for access, networking, and vulnerability or artifact governance integrations.

21. How does an AKS cluster pull a private image from ACR?

Answer: The kubelet or configured registry authentication path needs the appropriate pull permissions, and registry DNS/network access must work. Check image name, tag/digest, token permissions and private endpoints.

22. What is Microsoft Entra Workload ID for AKS?

Answer: An identity federation mechanism that lets a Kubernetes service account exchange an OIDC token for Azure access without embedding long-lived Azure credentials in the pod.

23. How would a pod access Azure Key Vault securely?

Answer: Use Workload ID with scoped authorization and appropriate Azure SDK support; verify service account annotation, OIDC federation, Key Vault permissions, DNS, firewall and private endpoint routing.

24. What is the difference between Kubernetes RBAC and Azure RBAC?

Answer: Kubernetes RBAC limits actions against Kubernetes API resources. Azure RBAC controls Azure resources; AKS also supports an Azure RBAC mode for Kubernetes authorization.

25. What are Azure CNI Overlay and Azure CNI Pod Subnet?

Answer: Overlay allocates pod addresses from a separate CIDR and conserves VNet IPs. Pod Subnet assigns pod addresses from VNet subnets and needs careful VNet IP sizing.

26. What would you monitor for AKS?

Answer: Control-plane/workload events, node CPU/memory/disk, pod restarts, availability, latency, error rates, ingress response codes, autoscaling, security alerts and container logs using Azure Monitor or Prometheus-compatible tools.

27. How would you plan an AKS upgrade?

Answer: Check supported versions and deprecated APIs; test application compatibility, PDBs, autoscaling and capacity; upgrade through a documented plan; verify rollout, ingress, storage and SLOs.

Useful Azure CLI checks

```
az login
az account list --output table
az account set --subscription "<subscription-id>"
az aks list --output table
az aks show -g "<resource-group>" -n "<aks-cluster>" -o table
az aks get-credentials -g "<resource-group>" -n "<aks-cluster>"
az acr repository list --name "<acr-name>" -o table
kubectl config current-context
```

Note: az aks get-credentials updates the local kubeconfig. Verify the selected subscription, context and namespace before any state-changing command.

5. Kubernetes fundamentals - questions and answers

28. What is a pod, deployment, service and ingress?

Answer: A pod is the smallest deployable scheduling unit; a Deployment manages replicas and updates; a Service provides stable discovery/load distribution; an Ingress routes HTTP(S) when a compatible ingress controller is installed.

29. What is a ReplicaSet?

Answer: A controller that maintains a desired number of pod replicas. Deployments normally create and manage ReplicaSets for rolling updates and history.

30. What is the difference between ClusterIP, NodePort and LoadBalancer services?

Answer: ClusterIP is cluster-internal; NodePort exposes a port on nodes; LoadBalancer asks a supported cloud/controller integration to provision an external or internal load balancer.

31. What are readiness, liveness and startup probes?

Answer: Readiness determines whether a pod receives service traffic; liveness triggers container restart on persistent failure; startup defers other probes until a slow-starting application is ready.

32. Why use resource requests and limits?

Answer: Requests influence scheduling and resource guarantees; limits cap usage. CPU throttling and memory OOM kills are possible when limits are unsuitable.

33. What is the difference between ConfigMap and Secret?

Answer: Both provide configuration, but Secrets are intended for sensitive values and require access controls, encryption and careful handling; base64-encoded Secret data is not encryption.

34. When would you use a StatefulSet instead of a Deployment?

Answer: For workloads needing stable pod identities and persistent volume associations, such as certain clustered databases or message systems.

35. What are PV, PVC and StorageClass?

Answer: A PersistentVolume provides storage, a PVC requests storage, and a StorageClass defines dynamic provisioning properties and supported volume types.

36. What are labels, selectors, taints and tolerations?

Answer: Labels identify resources; selectors choose them. Taints repel pods unless they tolerate them, supporting separation of node types or workloads.

37. How does a rolling update work?

Answer: A Deployment gradually replaces old pods with new ones according to its rollout strategy; readiness and replica capacity are important for continuity.

38. How do namespaces, RBAC and NetworkPolicies differ?

Answer: Namespaces organize resources; RBAC restricts Kubernetes API actions; NetworkPolicies control allowed pod traffic when the chosen CNI/network policy engine enforces them.

39. What is Helm?

Answer: A Kubernetes package manager that uses charts (templates and values) to create, version, install and upgrade related manifests.

40. What is the Horizontal Pod Autoscaler (HPA)?

Answer: A controller that adjusts replica counts based on supported resource or custom metrics, subject to scaling limits and capacity.

41. What is the difference between a Kubernetes context and namespace?

Answer: A context references a cluster, user and default namespace; a namespace is a resource scope within a cluster. Always verify both before issuing changes.

6. Terraform and infrastructure as code - questions and answers

42. What does Terraform do?

Answer: It declares desired infrastructure and uses providers to compare configuration with state and real resources, producing a plan of changes before apply.

43. Explain init, fmt, validate, plan and apply.

Answer: init installs providers/modules and prepares backends; fmt formats HCL; validate checks configuration; plan proposes changes; apply makes reviewed changes.

44. Why store Terraform state remotely?

Answer: So team members and CI systems share consistent infrastructure state with controlled access, locking and recovery; state may include sensitive data.

45. What is a Terraform module?

Answer: A reusable collection of resource definitions, variables and outputs. Modules standardize patterns such as Azure networking or AKS clusters.

46. How do you separate dev, test and production?

Answer: Use clearly separated configurations/backends, subscriptions or resource groups as appropriate, controlled variables, credentials and approval policies.

47. What is configuration drift?

Answer: A mismatch between deployed infrastructure and declared configuration/state, often caused by manual changes. Investigate and reconcile intentionally rather than blindly overwriting.

48. What happens if a Terraform apply fails halfway?

Answer: Some resources may already be created or changed. Inspect output and refreshed state, then create and review a new plan; do not assume a transaction-wide rollback.

49. How do you protect Terraform in CI/CD?

Answer: Use least-privilege identity federation, policy checks and review of saved plans; secure state, restrict apply permissions, pin providers/modules and require approvals for production.

Useful Terraform commands

```
terraform fmt -check -recursive
terraform init
terraform validate
terraform plan -out=tfplan
terraform show tfplan
# Apply only an approved, trusted plan in a controlled environment:
terraform apply tfplan
terraform state list
terraform output
```

Do not check tfplan files, tfstate, credentials, or secret-bearing outputs into source control. Production applies should run through a reviewed and authorized workflow.

7. DevSecOps and security fundamentals - questions and answers

50. What does DevSecOps mean?

Answer: Integrating security controls and feedback into design, coding, build, release and operations instead of adding security checks only before production.

51. What is SAST vs DAST vs SCA?

Answer: SAST examines source/code representations; DAST examines a running application; SCA identifies vulnerable or risky third-party dependencies and often license issues.

52. What does a container image scanner detect?

Answer: Known issues in image packages/dependencies and sometimes misconfiguration; it does not prove an image is trustworthy or free of exploitable defects.

53. What is the purpose of secret scanning?

Answer: To detect accidental exposure of passwords, tokens or keys in code/history. If a key leaks, revoke or rotate it and investigate, not merely delete the latest commit.

54. What is an SBOM?

Answer: A software bill of materials listing components/dependencies used to build software, helpful for inventory, compliance and vulnerability response.

55. Why sign images and verify provenance?

Answer: To establish that artifacts came from trusted build processes and have not been substituted, complementing vulnerability scans.

56. What does least privilege mean for CI/CD?

Answer: Give each build/deploy identity only the permissions and duration needed, limit production authority, use OIDC and protect approval boundaries.

57. How do you handle a critical vulnerability with no patch?

Answer: Validate affected versions and exposure, assess exploitability, apply compensating controls, document risk ownership, monitor and schedule remediation/exception expiry.

58. What are policy-as-code and admission controls?

Answer: Automated rules checking configurations and/or blocking deployments that violate policies such as privileged pods, untrusted images or mandatory labels.

59. How should sensitive values reach a Kubernetes application?

Answer: Prefer a centrally managed secret store and workload identity. Avoid plaintext values in repositories, command logs, image layers and unrestricted manifests.

8. Observability and incident response - questions and answers

60. What are logs, metrics and traces?

Answer: Logs describe discrete events; metrics measure numeric signals over time; traces show the path of requests across services.

61. What does 99.9% availability mean?

Answer: For a simple time-based 30-day period, a 0.1% downtime allowance is about 43.2 minutes. Request-based SLOs use a different denominator.

62. What is the difference between MTTD and MTTR?

Answer: Mean time to detect measures detection delay; mean time to restore/recover measures recovery duration. Define calculation boundaries consistently.

63. What are the first steps during a production incident?

Answer: Assess scope, stop risky changes, check recent deploys and telemetry, gather evidence, choose safe remediation, communicate status, validate recovery and document follow-up.

64. How do you choose rollback versus roll-forward?

Answer: Consider blast radius, time-to-recovery, schema/data compatibility, confidence in fix and ability to verify. Preserve evidence and avoid introducing more changes than necessary.

9. Essential kubectl command reference

Replace values inside angle brackets. Commands default to the selected context and namespace unless -n or -A is used. Most commands below inspect state; rollout, scale and patch commands modify workloads.

Confirm environment and resources

Command	Purpose
`kubectl config get-contexts`	List available kubeconfig contexts.
`kubectl config current-context`	Show current context before acting.
`kubectl config use-context <context>`	Switch to a specified cluster context.
`kubectl get namespaces`	List namespaces.
`kubectl get nodes -o wide`	List nodes and their state/IP.
`kubectl describe node <node>`	Investigate node conditions and allocations.
`kubectl get pods -A -o wide`	Find pods across all namespaces and nodes.
`kubectl get all -n <ns>`	Quick view of common workload objects.
`kubectl get events -n <ns> --sort-by=.lastTimestamp`	Read recent namespace events.

Investigate an application

Command	Purpose
`kubectl get deploy -n <ns>`	Check Deployments and replica readiness.
`kubectl describe deploy <name> -n <ns>`	Inspect deployment conditions and events.
`kubectl describe pod <pod> -n <ns>`	Review pod scheduling, restarts and container states.
`kubectl logs <pod> -n <ns>`	Read logs from a pod.
`kubectl logs <pod> -c <container> -n <ns>`	Logs for a specific container.
`kubectl logs <pod> -n <ns> --previous`	Read logs from previous terminated container.
`kubectl logs -f <pod> -n <ns>`	Follow logs live.
`kubectl top pods -n <ns>`	Check resource use (metrics API required).
`kubectl top nodes`	Check node CPU/memory usage.
`kubectl exec -it <pod> -n <ns> -- sh`	Open shell if image contains one and policy allows.

Network and service routing

Command	Purpose
`kubectl get svc -n <ns>`	Inspect Services, ports and IPs.
`kubectl describe svc <svc> -n <ns>`	View selector and endpoints.
`kubectl get endpointslice -n <ns>`	Check backend endpoints.
`kubectl get ingress -n <ns>`	Inspect ingress routes.
`kubectl describe ingress <name> -n <ns>`	Review ingress rules and controller events.
`kubectl get networkpolicy -n <ns>`	List applicable NetworkPolicies.
`kubectl port-forward -n <ns> svc/<svc> 8080:80`	Temporarily forward a local port for testing.

Rollouts, configuration and storage

Command	Purpose
`kubectl rollout status deployment/<name> -n <ns>`	Check rollout success/failure.
`kubectl rollout history deployment/<name> -n <ns>`	Show available revision history.
`kubectl rollout undo deployment/<name> -n <ns>`	Roll back one rollout revision (changes workload).
`kubectl scale deployment/<name> --replicas=3 -n <ns>`	Set replica count (changes workload).
`kubectl get hpa -n <ns>`	Inspect autoscaling status.
`kubectl get configmap -n <ns>`	List ConfigMaps.
`kubectl get secret -n <ns>`	List secret names/types only; avoid printing values.
`kubectl get pvc -n <ns>`	Inspect volume claims.
`kubectl describe pvc <pvc> -n <ns>`	Investigate storage provisioning and events.
`kubectl get pv`	Inspect cluster PersistentVolumes.
`kubectl get storageclass`	View available storage classes.

Access and deployment tooling

Command	Purpose
`kubectl auth can-i create deployments -n <ns>`	Check current identity authorization.
`kubectl get serviceaccount -n <ns>`	List Kubernetes service accounts.
`kubectl get role,rolebinding -n <ns>`	Review namespace-scoped Kubernetes RBAC.
`helm list -n <ns>`	List Helm releases in namespace.
`helm status <release> -n <ns>`	Inspect Helm release status.
`helm history <release> -n <ns>`	Inspect Helm revision history.
`helm template <release> ./chart -f values.yaml`	Render a local chart before deployment.
`helm lint ./chart`	Validate common Helm chart problems.

10. Common Kubernetes errors: what they mean and what to check

Symptom	Meaning	First investigation
CrashLoopBackOff	Container repeatedly exits or gets restarted.	describe pod; logs --previous; app config, OOM, probes, dependencies.
ImagePullBackOff / ErrImagePull	Image cannot be pulled.	Image/tag/digest; ACR permissions; DNS/firewall; registry availability.
Pending	Pod not scheduled or waiting for a requirement.	Pod events; requests; taints; node selectors; quota; PVC; autoscaler.
OOMKilled	Container terminated after memory exhaustion.	describe pod; memory limit/use; app heap/leaks; workload sizing.
CreateContainerConfigError	Container configuration cannot be assembled.	Missing Secret/ConfigMap keys; invalid env/volume reference.
CreateContainerError	Runtime failed to create container.	Container config, node runtime, mounts, image and events.
ContainerCreating stuck	Provisioning, image, network or mount still pending.	describe events; CNI and CSI logs; volume attach; registry access.
Readiness probe failed	Pod not considered ready for Service traffic.	Endpoint path, port, timeout, app startup, network/policy.
Liveness probe failed	Probe indicates unhealthy container; restarts may follow.	Probe path and timing; slow startup; application health.
HTTP 503 from ingress	No usable backend or upstream error.	Ingress rules, Service ports, EndpointSlices, readiness, app logs.
PVC Pending	Claim cannot bind/provision storage.	StorageClass; access mode; capacity; CSI; topology/quotas.
FailedMount / FailedAttachVolume	Volume mount or attachment could not complete.	CSI events; secrets; zone; existing attachment; permissions.
Forbidden	Caller lacks Kubernetes API permission.	kubectl auth can-i; Role/RoleBinding or configured AKS authorization.
Node NotReady	Node cannot report healthy status.	Node describe; kubelet/runtime; network/CNI; disk/memory pressure.
DNS resolution failure	Pod cannot resolve a service/hostname.	CoreDNS, DNS policies, network policies, resolver settings.
Unschedulable GPU pod	GPU requirements cannot be placed.	GPU node pool, resource requests, device plugin, taints and quota.

11. Scenario exercises with investigation paths

S1. Pods restart after a new release

Symptom: A deployment starts returning errors and pods repeatedly restart.

First checks: kubectl rollout status; describe pod; logs --previous; review probes, resource limits, new ConfigMaps/Secrets and image digest.

Reasoning / recovery: Separate application failures from platform problems. Roll back an incompatible image if safe, verify service health and find the underlying change.

S2. Pod cannot download an image from ACR

Symptom: Pods show ImagePullBackOff following a registry or identity change.

First checks: kubectl describe pod; verify image digest/tag, ACR repository, kubelet pull authorization, private endpoints and DNS.

Reasoning / recovery: Fix the specific image, access or network issue. Avoid adding static registry passwords when managed identity is the intended approach.

S3. Pending pods despite autoscaling

Symptom: New pods remain Pending for 20 minutes.

First checks: kubectl describe pod; kubectl get nodes; inspect pod selectors, affinity, PVC zone, node pool max, Azure VM quota and cluster autoscaler events.

Reasoning / recovery: Confirm whether an added node can satisfy the pod; resolve the scheduling constraint or safely adjust capacity.

S4. Ingress returns 503

Symptom: User traffic fails while containers appear Running.

First checks: kubectl get ingress,svc,endpointslice,pods; describe ingress; readiness, selector and targetPort; ingress-controller logs.

Reasoning / recovery: Follow the path from ingress to Service to ready endpoints. Fix selector/port/probe/routing problem rather than assuming the public load balancer is broken.

S5. Azure Key Vault authorization failure

Symptom: A pod gets HTTP 403 from Key Vault after deployment.

First checks: Check Workload ID service account, federated credential subject/audience, Azure role or Key Vault access policy, token errors and network access.

Reasoning / recovery: Correct identity authorization separately from networking/DNS. Do not put broad Azure credentials into the pod.

S6. Terraform plan deletes a resource unexpectedly

Symptom: A pipeline proposes destroying a production storage resource.

First checks: Review plan, changed configuration, state location/workspace, provider versions, drift and resource naming; stop before apply.

Reasoning / recovery: Investigate state/address changes, import or moved blocks where warranted, review with owner and obtain explicit approval before remediation.

S7. GitHub Actions succeeds but deployment fails

Symptom: CI tests pass, but deploy job cannot update AKS.

First checks: Inspect workflow permissions, environment approval, OIDC id-token permission, Azure federated identity/RBAC and kubeconfig context.

Reasoning / recovery: Use clear separate build and deploy identities; repair missing scope/authorization and verify the exact target cluster.

S8. Application slow after scale-out

Symptom: More replicas did not reduce API latency.

First checks: Check p95 latency, saturation, downstream database, thread pools, CPU throttling, HPA metrics, ingress and service routing.

Reasoning / recovery: Determine the bottleneck; scaling pods alone may not resolve database locks or shared dependencies.

S9. A vulnerable image is already running

Symptom: Security scan finds a critical issue in a production image.

First checks: Confirm impacted image digest, exposed pods/namespaces, exploitability, vendor fix and runtime telemetry.

Reasoning / recovery: Coordinate risk response; rebuild and promote a fixed image, or use documented compensating controls with monitoring and expiration.

S10. Secret accidentally committed

Symptom: A developer pushes a cloud credential into the repository.

First checks: Identify scope and whether token is still valid; consult audit logs; locate affected history, logs and built images.

Reasoning / recovery: Immediately revoke/rotate, investigate exposure, remove unsafe copies where feasible, and add preventive scanning and federation.

S11. AKS node upgrade causes disruption

Symptom: Several pods become unavailable during node pool maintenance.

First checks: Inspect node drain events, readiness, PodDisruptionBudgets, spare capacity, volume topology, probes and replicas.

Reasoning / recovery: Restore capacity and reschedule safely; adjust PDBs, surge and spread strategy before resuming the upgrade.

S12. DNS fails only for a new namespace

Symptom: Pods resolve public domains incorrectly or cannot find internal services.

First checks: Check CoreDNS, namespace-level NetworkPolicy egress to DNS, DNS configuration, FQDN and service namespaces.

Reasoning / recovery: Test name resolution from a permitted troubleshooting pod; fix narrowly scoped policies or DNS configuration.

12. Short hands-on practice exercises

1. Kubernetes rollout

Task: Deploy a sample NGINX application with three replicas and a ClusterIP Service. Set readiness/liveness probes and resource requests.

Success check: Demonstrate kubectl describe, rollout status, pod logs, endpoints and a controlled rollout/undo.

2. Helm customization

Task: Create or modify a chart with image repository/tag, replicas, service port and namespace-specific values.

Success check: helm lint and helm template pass; values change the rendered manifests as expected.

3. GitHub Actions CI

Task: Create a pull-request workflow that checks out code, installs dependencies, runs tests and builds an artifact.

Success check: A failing test blocks the workflow; production deployment is not triggered from an untrusted pull request.

4. Terraform safety

Task: Use a disposable Azure sandbox to plan a small resource group/network deployment, storing state securely.

Success check: Show fmt, validate and plan; explain state locking, permissions, cost and cleanup before any apply.

5. DevSecOps gate

Task: Add a dependency or image scan to a demo pipeline and define a documented threshold for release blocking.

Success check: Explain false positives, triage, exception ownership and how to keep reporting reproducible.

6. Troubleshooting drill

Task: Introduce a wrong Service targetPort or missing ConfigMap key into a disposable Kubernetes namespace.

Success check: Diagnose through events, describe, endpoints and logs; explain the root cause and corrective change.

13. Last-minute review checklist

• Explain the complete flow from Git commit through CI, image scanning, image registry, approval, Helm/AKS deployment and production validation.

• Be able to draw an AKS architecture including VNet, ingress, ACR, Key Vault, identity, monitoring and network restrictions.

• Know how to diagnose Pending, CrashLoopBackOff, ImagePullBackOff, OOMKilled, 503, PVC Pending and Forbidden errors.

• Know kubectl describe, logs --previous, get events, rollout status, get svc/endpointslice, top, and auth can-i.

• Explain GitHub Actions workflow syntax, job dependencies, secret handling, production environments and Azure OIDC federation.

• Explain Terraform state, remote backends, init/plan/apply, drift, modules and why a plan must be reviewed.

• Know SAST, DAST, SCA, container scanning, signing, SBOM, least privilege and incident response for leaked secrets.

• Always check cluster context, environment and change approval before running modifying commands.

End of preparation guide.